ONNX Runtime 1.24.4 量化推理支持与架构优化报告

> 分析范围：CPU/GPU 后端，x86/ARM/RISC-V 三大架构，INT4/INT8 推理能力 数据来源：`~/onnxruntime` 源码（版本 1.24.4）

摘要

本报告对 ONNX Runtime 1.24.4 的量化推理能力进行了全面的源码级分析，覆盖 INT8 和 INT4 两条技术路径，横跨 x86、ARM、RISC-V 三大 CPU 架构以及 CUDA/TensorRT GPU 后端。核心发现如下：

INT8 方面：x86（VNNI/AMX）和 ARM（SDOT/SMMLA）拥有成熟的 INT8 推理生态——从专用 MAC 指令到完整的 MLAS GEMM/Conv/DWConv kernel，形成自动分发的梯度优化链。**RISC-V RVV 1.0** 完全空白：无 `MLAS_TARGET_RISCV` 宏、无优化 kernel、无平台检测，所有 INT8 算子 fallback 到 scalar C 代码。根本原因是 RVV 缺少分段规约指令（x86 的 `vpdpbusd` / ARM 的 `sdot` 能一条指令将 4×i8 点积压缩为 1×i32 且多 lane 并行输出，而 RVV 只有全向量 `vredsum` 归约为 1 个标量）。报告定量分析了 RVV 外积法 fallback（约 ARM SDOT 的 30-50%）和自定义 `vsegdot` 分段点积指令的潜在收益（可提升至 ARM SDOT 的 85-100%，约 2-3× 加速）。

INT4 方面：INT4 在 CPU/GPU 后端均无卷积算子，只有 MatMul 路径（专供 LLM 的 linear layer）。INT4 实际计算路径是"解包→转 FP32→fmadd"，每值指令数是 INT8 的 4 倍，仅在 memory-bound 场景（LLM 单 batch）有优势。YOLOv11 等 CNN 模型无法使用 INT4 推理。

算子选择方面：只有 MAC 密集型算子（Conv/DWConv/MatMul，占 YOLOv11 推理 95%+ FLOPs）值得做 INT8 量化，因为 INT8 的 4× MAC 吞吐优势能充分发挥。SiLU/NMS 等算子不做 INT8 不是因为"做不了"，而是做了没收益——它们是 memory-bound 或非 MAC 型操作，INT8 加速机制不适用。这是所有主流框架（TensorRT/TFLite/OpenVINO/NCNN/TVM/SNPE）的共同结论。

报告结构：第一～三章梳理 ORT 量化算子支持现状；第四章深入分析 RVV 分段规约缺陷及其对 GEMM kernel 的影响；第五章对比 INT4 与 INT8 的实际计算路径和适用场景；第六～八章详解 CPU/GPU 各架构的 kernel 实现；第九章分析 YOLOv11 INT8 管线及算子选择逻辑；第十章提出 RVV 补全方案（外积 fallback + 自定义 `vsegdot` 指令）。

一、总览

维度	x86	ARM	RISC-V
INT8 专用指令	VNNI / AVX-VNNI / AMX-INT8	SDOT / UDOT / SMMLA / UMMLA	无专用指令，需 RVV 组合
INT8 MLAS Kernel	完整（SSE → AVX2 → VNNI → AMX）	完整（NEON → SDOT → SMMLA）	无（scalar fallback）
INT4 MLAS Kernel	AVX512 专用 kernel	NEON + I8MM 辅助	无
平台检测与分发	CPUID 自动检测	CPU feature register 检测	无检测逻辑
YOLOv11 INT8 可行性	高（Conv/DWConv 完整）	高	极低（全部 scalar）
分段规约能力	✅ 硬件内建（VNNI 4→1）	✅ 硬件内建（SDOT 4→1）	❌ 无，需 vredsum 串行规约

二、ONNX Runtime 量化算子支持清单

2.1 INT8 算子支持

算子	CPU 后端	CUDA 后端	TensorRT	说明
QLinearConv	✅ 支持 grouped/DWConv	✅ cuDNN/cuBLAS	✅	核心 INT8 卷积，含深度可分离路径
QLinearMatMul	✅	✅ cuBLAS `cublasGemmEx`	✅	INT8 矩阵乘
ConvInteger	✅	✅	✅	整数卷积（不含 scale/zp）

算子	CPU 后端	CUDA 后端	TensorRT	说明
MatMulInteger	✅	✅ `matmul_integer.cu`	✅	整数矩阵乘
QuantizeLinear	✅ 支持 int8/uint8/int4	✅ `quantize_linear.cu`	✅	FP→INT 量化，含 per-channel
DequantizeLinear	✅ 支持 int8/uint8/int4	✅	✅	INT→FP 反量化
DynamicQuantizeLinear	✅	✅	—	运行时动态量化
QLinearAdd	✅	—	—	INT8 加法
QLinearMul	✅	—	—	INT8 乘法
QLinearAveragePool	✅	—	—	INT8 平均池化
QLinearConcat	✅	—	—	INT8 拼接
QLinearLeakyRelu	✅	—	—	INT8 LeakyReLU
QLinearSigmoid	✅	—	—	INT8 Sigmoid
QLinearGlobalAveragePool	✅	—	—	INT8 全局平均池化

注意：`QLinearSigmoid` 存在但 `QLinearSiLU` 不存在。YOLOv11 大量使用 SiLU，是 INT8 管线的主要断点。

2.2 INT4 算子支持

算子	CPU 后端	CUDA 后端	说明
MatMulBnb4			FP4/NF4 权重矩阵乘

算子	CPU 后端	CUDA 后端	说明
	✅ `contrib_ops/cpu/`	✅ `contrib_ops/cuda/` sm_80+	（bitsandbytes 格式）
MatMulNbits	✅ `contrib_ops/cpu/`	—	N-bit 通用量化矩阵乘
MatMulFpQ4	✅ `contrib_ops/cpu/`	—	FP32 × Q4(BlockInt4) 矩阵乘
DequantizeBlockwise4Bits	✅	✅ sm_53+	Block-wise 4-bit 反量化
DequantizeLinear (int4)	✅	✅	int4_t/uint4_t 模板特化
QuantizeLinear (int4)	✅	✅	int4_t/uint4_t 模板特化
FP16+INT4 混合精度	—	✅ sm_80+ `llm/fpA_intB_gemm/`	FP16 激活 × INT4 权重
BF16+INT4 混合精度	—	✅ sm_80+	BF16 激活 × INT4 权重

2.3 INT4 算子覆盖的关键空白：无卷积

ONNX Runtime 中 **INT4** 完全没有卷积类算子，**CPU** 和 **GPU** 后端均如此。

算子类型	CPU INT8	GPU INT8	CPU INT4	GPU INT4
MatMul	✅ QLinearMatMul	✅ cuBLAS	✅ MatMulBnb4 / MatMulFpQ4	✅ CUDA kernel
Conv	✅ QLinearConv	✅ cuDNN	❌ 不存在	❌ 不存在

算子类型	CPU INT8	GPU INT8	CPU INT4	GPU INT4
DWConv	✅ 专用 kernel	✅	❌ 不存在	❌ 不存在

INT4 算子全部是 矩阵乘法（MatMul 系），专门服务 LLM 的 linear layer。卷积类算子（标准 Conv、DWConv、PWConv）在 INT4 下全平台空白。

这意味着 **YOLOv11** 无法使用 **INT4** 推理——其主体计算是 Conv/DWConv，INT4 下只能走"反量化→FP32 Conv"的迂回路径，开销比直接 INT8 更大。

三、INT8 专用加速指令集对比

3.1 三大架构的 INT8 加速指令

架构	指令	引入版本	功能	吞吐
x86	`VPDPBUSDB` (VNNI)	AVX512-VNNI / AVX-VNNI	4× `uint8 × int8` → `int32` 累加	512-bit: 32 个乘加/cycle
x86	`TDPBSSD` (AMX)	AMX-INT8 (Sapphire Rapids)	矩阵级 INT8 乘累加（16×16 tile）	硬件矩阵引擎
ARM	`SDOT`	ARMv8.4	4× `int8 × int8` → `int32` 累加	128-bit: 8 个乘加/cycle
ARM	`UDOT`	ARMv8.4	4× `uint8 × uint8` → `int32` 累加	同上
ARM	`SMMLA`	ARMv8.6 (I8MM)	8× `int8` 矩阵乘累加	128-bit: 16 个乘加/cycle
ARM	`UMMLA`	ARMv8.6 (I8MM)	8× `uint8` 矩阵乘累加	同上
	—	—		—

架构	指令	引入版本	功能	吞吐
RISC-V			无专用 **INT8** 指令	

3.2 指令本质：分段规约（Segmented Reduction）

VNNI、SDOT、SMMLA 的共同核心能力不仅是"乘累加"，更关键的是 分段规约——在一个寄存器内部，将相邻元素分组求和，产出多个独立结果：

```
VNNI `_mm512_dpbusd_epi32(dst, a, b)`:
  512-bit 寄存器 = 64 个 int8
  分成 16 组，每组 4 个 int8
  第0组: a[0]×b[0] + a[1]×b[1] + a[2]×b[2] + a[3]×b[3] → dst[0] (int32)
  第1组: a[4]×b[4] + a[5]×b[5] + a[6]×b[6] + a[7]×b[7] → dst[1] (int32)
  ...
  第15组: ...                                          → dst[15] (int32)

  一条指令 → 同时产出 16 个独立的 int32 结果
```

```
ARM SDOT `SDOT Vd.4S, Vn.16B, Vm.4B[Index]`:
  128-bit 寄存器 = 16 个 int8
  分成 4 组，每组 4 个 int8
  第0组: Vn.b[0:3] × Vm.b[0] → Vd.s[0] (int32)
  第1组: Vn.b[4:7] × Vm.b[1] → Vd.s[1] (int32)
  第2组: Vn.b[8:11] × Vm.b[2] → Vd.s[2] (int32)
  第3组: Vn.b[12:15] × Vm.b[3] → Vd.s[3] (int32)

  一条指令 → 同时产出 4 个独立的 int32 结果
```

这使得 **A**、**B** 的 **LOAD** 和 **C** 的 **STORE** 都能 **SIMD** 化：加载一个向量寄存器的 A 和 B，一条 VNNI/SDOT 就算出多个 C 元素，直接向量化写回。

四、RVV 的根本局限：缺少分段规约

4.1 问题本质

RVV 1.0 没有分段规约指令。它只有： - `vmacc.vv` ：逐元素乘累加（无横向求和） - `vwmacc.vv` ：逐元素拓宽乘累加（int8×int8→int16，无横向求和） - `vredsum.vs` ：全寄存器规约求和 → 只产出 **1** 个标量

```
 RVV 的困境：

vle8.v   vA, (A_ptr)       ; SIMD 加载 16 个 int8
vle8.v   vB, (B_ptr)       ; SIMD 加载 16 个 int8
vwmacc.vv vC, vA, vB       ; 16 个 int8 逐元素乘累加到 int16（无分组求和！）
```

```
                                ; 结果仍然是 16 个独立的 int16，不是 4 个 int32
vredsum.vs vC, vC, v0       ; 全寄存器规约 → 只得到 1 个 int32 标量！
```

一条 `vredsum` 只能产出 **1** 个 **int32** 结果，而不是 VNNI 的 16 个或 SDOT 的 4 个。

4.2 对 GEMM Kernel 数据流的影响

x86 VNNI 的 **GEMM** 内层循环（AVX2-VNNI，256-bit）：

```
; 一次迭代：同时更新多个 C[m][n]
vpbroadcastd ymm2, [rcx]           ; 加载 1 个 A 元素，广播到 8 个 lane
vmovdqu      ymm0, [rdx]           ; SIMD 加载 32 bytes B（= 8 组 × 4 int8）
vpdpbusd     ymm4, ymm2, ymm0      ; 一条指令 → 8 个 C 元素同时更新
                                       ; ymm4[0] = Σ(a×b[0:3])
                                       ; ymm4[1] = Σ(a×b[4:7])
                                       ; ...
                                       ; ymm4[7] = Σ(a×b[28:31])
; C 的 STORE：vmovdqu [rdi], ymm4   ; 8 个 int32 一次性写回 ✅ 全 SIMD
```

ARM SDOT 的 **GEMM** 内层循环：

```
ld1       {v0.16b}, [x1]           ; SIMD 加载 16 int8 B
SdotByElement 16, 0, 4, 0         ; v16 += dot(v0.b[0:3],  v4.b[0])
SdotByElement 18, 0, 4, 1         ; v18 += dot(v0.b[4:7],  v4.b[1])
SdotByElement 20, 0, 4, 2         ; v20 += dot(v0.b[8:11], v4.b[2])
SdotByElement 22, 0, 4, 3         ; v22 += dot(v0.b[12:15],v4.b[3])
; 4 个 C 元素并行更新，后续 st1 {v16-v22} 批量写回 ✅ 全 SIMD
```

RVV 被迫的 **GEMM** 内层循环：

```
// 方案 A：规约到标量，逐个 C 串行写回
vint8mf2_t vA = vle8_v_i8mf2(A_ptr, 16);    // SIMD 加载 A ✅
vint8mf2_t vB = vle8_v_i8mf2(B_ptr, 16);    // SIMD 加载 B ✅
vint32m1_t vC = vwmacc_vv_i32m1(vC, vA, vB); // 16 个 int8→int16，但无分组
vredsum     → 1 个 int32                        // ❌ 只产出 1 个结果
sw         C[0], vC_scalar                     // ❌ 串行写回 1 个元素

// 要计算 C[1]，必须重新加载不同的 B 行，再次规约
// 每个 C 元素都需要一次完整的 vredsum
```

4.3 数据流对比图

```
                    x86 VNNI / ARM SDOT                          RISC-V RVV
                ┌──────────────────────┐                ┌──────────────────────┐
  LOAD A        │ vpbroadcast / ld1    │                │ vle8.v               │
  (1 element)   │ (SIMD, 1条指令)      │                │ (SIMD, 1条指令)      │
```

```
                    │                                   │
                    │                                   │
LOAD B              ▼                                   ▼
(16 int8)   vmovdqu / ld1                       vle8.v
            (SIMD, 1条指令)                     (SIMD, 1条指令)
                    │                                   │
                    │                                   │
COMPUTE             ▼                                   ▼
            vpdpbusd / SDOT                     vwmacc.vv
            1条指令 → 4~16个int32               逐元素乘累加
            [内置分段规约]                      仍是 16 个独立值
                    │                                   │
                    │                                   │
                    │                                   ▼
                    │                           vredsum.vs
                    │                           全寄存器规约
                    │                           → 只有 1 个 int32
                    │                                   │
                    │                                   │
STORE C             ▼                                   ▼
            vmovdqu / st1                       sw (scalar store)
            4~16个int32 一次写回                只能写回 1 个 int32
            ✅ SIMD STORE                       ❌ 串行 STORE
```

4.4 性能影响量化

操作	**x86 VNNI (256-bit)**	**ARM SDOT (128-bit)**	**RVV (128-bit)**
LOAD A	1 指令（broadcast）	1 指令（broadcast）	1 指令（vle8）
LOAD B	1 指令（32 bytes）	1 指令（16 bytes）	1 指令（16 bytes）
乘累加	1 指令（VPDP / SDOT）	1 指令（SDOT）	1 指令（vwmacc）
分段规约	内含在乘累加指令中	内含在乘累加指令中	无，需额外 **vredsum**
横向规约	不需要	不需要	1+ 指令（vredsum）
产出 **C** 元素数	**8** 个	**4** 个	**1** 个
STORE	**1** 条 **SIMD** 指令	**1** 条 **SIMD** 指令	**1** 条标量 **sw**

关键指标——每对 **A/B** 加载产出的有效 **C** 元素数： - x86 VNNI：**8** 个 int32 / 次加载 - ARM SDOT：**4** 个 int32 / 次加载 - RVV：**1** 个 int32 / 次加载

这意味着 RVV 需要比 VNNI 多 8 倍的迭代次数才能算出同样多的 C 元素，且每次迭代末尾都有一个串行的 vredsum + scalar store。

4.5 x86/ARM INT8 GEMM Kernel 三大核心操作拆解

通过对 MLAS 源码的深入分析，INT8 GEMM microkernel 的高效依赖于三个紧密耦合的操作。**RVV** 在这三层上都存在缺陷：

操作 **1**：**PackB** 交错打包（**Interleave Pack**）

x86/ARM 在计算前对 B 矩阵做预处理，将多行数据交错排列，使同一列的 4 个元素连续存储——恰好是 `vpdpbusd` / `sdot` 一个 32-bit lane 所需的对齐格式。

x86 AVX2 CopyPackB（`QgemmU8S8KernelAvx2.asm:596-606`）：

```
原始 B（4 行 × 16 列，行主序）：
  Row0: [b00 b01 b02 ... b0f]
  Row1: [b10 b11 b12 ... b1f]
  Row2: [b20 b21 b22 ... b2f]
  Row3: [b30 b31 b32 ... b3f]

经 vpunpcklbw → vpunpckhbw → vpunpcklwd → vpunpckhwd → vinserti128 交错后：
  [b00 b10 b20 b30 | b01 b11 b21 b31 | b02 b12 b22 b32 | b03 b13 b23 b33 | ...]
  [b04 b14 b24 b34 | b05 b15 b25 b35 | ...                                | b07 b17 b27 b
```

每 4 字节 = 同一列的 4 行数据 = `vpdpbusd` 一个 lane 的输入。

ARM SDOT CopyPackB（`qgemm_kernel_sdot.cpp:538-548`）：

```
// 4 行 × 8 列 交错打包为 2 个 128-bit 向量
int8x16_t v02 = vcombine_s8(BytesRow[0], BytesRow[2]);
int8x16_t v13 = vcombine_s8(BytesRow[1], BytesRow[3]);
int8x16x2_t zw = vzipq_s8(v02, v13);           // 字节级交错
int16x8x2_t zd = vzipq_s16(...);                // 半字级再交错
```

RVV 缺失：无高效的 `vzip` / `vpunpck` 等价指令。需要多条 `vrgather` 或 `vslide` 组合模拟。

操作 **2**：**Kernel** 内循环——融合分段点积

x86 VNNI（`vpdpbusd ymm_dst, ymm_A_broadcast, ymm_B_packed`）：- A：broadcast 4 字节到整个 YMM（256-bit） - B：已打包，32 字节包含 8

列 × 4 行交错数据 - 一条指令：32 × i8 × 32 × i8 → 8 个 i32（每 lane 4 对乘加） - 输出直接是 8 个独立的 i32 累积器

ARM SDOT（`SdotByElement dst, B_vec, A_vec, index`）（`QgemmS8S8KernelSdot.asm:234-253`）：

```
SdotByElement 16, 0, 4, 0    ; v16 += dot(v0.b[0:3],  v4.b[0])
SdotByElement 18, 0, 4, 1    ; v18 += dot(v0.b[4:7],  v4.b[1])
SdotByElement 20, 0, 4, 2    ; v20 += dot(v0.b[8:11], v4.b[2])
SdotByElement 22, 0, 4, 3    ; v22 += dot(v0.b[12:15],v4.b[3])
; 4 个 C 元素并行更新，后续 st1 {v16-v22} 批量写回 ✅
```

操作 **3**：**4×4/4×8** 小矩阵乘展开

M8 kernel 每次处理 8×8 的 C 块，每个 K 迭代展开为： - 加载 4 个 A 向量（8 行 × PackedK 个元素） - 加载 4 个 B 向量（8 列 × PackedK 行的交错数据） - 执行 **32** 条 **sdot** 指令（8 行 × 4 个 B 向量） - 产出 32 个 i32 累积值 → 直接映射到 C 矩阵的 8×4 块

这三层操作形成完整的内积 **GEMM** 路径，全部依赖 `sdot` / `vpdpbusd` 的分段规约能力。

4.6 RVV 外积路径 vs 内积路径：量化对比

RVV 没有分段规约指令，但可以通过 外积法 绕开。以下对比三条路径的实际效率。

内积路径（有 **`sdot` / `vpdpbusd`**）— **ARM** 实际实现

```
每 K 迭代（4 个 K 元素）：
  Load A: 4 × 16B = 64B  (8行 × 2向量，每个含4个K元素)
  Load B: 4 × 16B = 64B  (8列 × PackedK的交错数据)
  sdot:   32 条指令
  产出:   8×8=64 个 i32 累积值（都在寄存器中，无需store）

  算术强度: 32 次乘加 / 128B = 0.25 ops/byte per K iter
  寄存器效率: 32 个 sdot → 64 个 i32 输出 → 100% 向量化输出
```

外积路径（**RVV** 现有指令，**VLEN=256**）

```
每 K 迭代（4 个 K 元素）：
  Load A: 1 个 i8 broadcast → 用 vrgather/vmv.s.x 展开
  Load B: 32B = 32 个 i8 (1 行 K × N列)
  vwmul:  32×i8 → 32×i16 (widening)
  vwadd:  32×i16 → 32×i32 (widening acc into C)
  产出:   32 个 i32 直接写入 C 的一行
```

```
算术强度: 32 次乘加 / 36B ≈ 0.89 ops/byte per K iter per row
但需要 8 行 × (256/4)=64 次 K 迭代, 总共 512 次 broadcast
```

定量估算

指标	内积（有分段规约）	外积（**RVV** 现有）	比值
每 **K** 迭代指令数	32 条 sdot	~64 条（8 行 × 8 条）	2×
C 输出向量化率	100%（8×8 块全在寄存器）	12.5%（每次只产出 1 行 × N 列）	8×
Load/Compute 比	128B load → 64 i32 out	36B load → 32 i32 out per row	—
寄存器压力	16 个 128-bit 足够	需要 LMUL≥4 才能一次算 8 行	—
预估整体吞吐	基准 **1.0×**	**~0.3-0.5×**	**2-3×** 差距

关键瓶颈：外积路径的 C 输出是逐行的，8 行需要 8 轮独立的 broadcast+乘加循环。内积路径通过 `sdot` 一条指令同时推进 8 行 × 4 列的计算，计算密度高 2-3 倍。

自定义 **`vsegdot`** 指令后的预估

```
有 vsegdot 的 RVV 内积路径:
  Load A: 64B (8行×8个K元素, packed)
  Load B: 64B (8列×8个K行, interleaved)
  vsegdot: 32 条 (等效 ARM sdot)
  产出:   64 个 i32

  预估吞吐: ~0.85-1.0× (接近 ARM SDOT)
```

结论：自定义 `vsegdot` 分段点积指令可带来约 **2-3×** 的 **INT8 GEMM** 吞吐提升（相比 RVV 外积路径），使 RVV INT8 性能接近 ARM SDOT 水平。

4.7 RVV 需要补充的自定义指令清单

优先级	指令	语义	收益
P0	`vsegdot.vv vd, vs1, vs2, e8, seg=4`	4×i8 点积→1×i32，多段并行，等价于 x86 `vpdpbusd` / ARM `sdot`	**INT8 GEMM** 吞吐 **2-3×**，解锁内积路径
P1	`vsegadd.vv vd, vs1, seg=4`	4×i8/i16 横向求和→1×i32/i64，等价于 x86 `vphaddd` / ARM `vpaddlq`	PackB 的 RowSum/ ColSum 加速，辅助规约
P2	`vinterleave vs1, vs2, stride`	多行数据交错打包，等价于 x86 `vpunpck*` / ARM `vzipq`	PackB 阶段 2-4× 加速，减少预处理开销

P0 的 **`vsegdot`** 是唯一决定性的指令——有了它就能写标准的 INT8 内积 GEMM kernel，与 x86 VNNI / ARM SDOT 同等架构。P1/P2 是优化项，可以在已有 RVV 指令上用 `vrgather` 等组合实现。

`vsegdot` 的伪代码语义：

```
vsegdot.vv vd, vs1, vs2, seg=4
  // vs1 = [a0 a1 a2 a3 | a4 a5 a6 a7 | ... ]  // VL × i8
  // vs2 = [b0 b1 b2 b3 | b4 b5 b6 b7 | ... ]  // VL × i8
  // vd.s[0] += a0*b0 + a1*b1 + a2*b2 + a3*b3
  // vd.s[1] += a4*b4 + a5*b5 + a6*b6 + b7*b7
  // ...
  // 产出 VL/4 个 i32 结果
```

五、INT4 vs INT8：额外指令开销与性能对比

5.1 INT4 到 INT8 的解包：不需要设计新指令

硬件层面没有原生 **INT4** 计算路径。CPU/GPU 里没有 4-bit 乘法器。但 INT4 解包到 INT8 完全不需要新指令，用的是所有 SIMD ISA 里最基础的位运算：

```
 INT4 packed 格式：一个 byte = [high_nibble | low_nibble]
例：0xAB = 存了两个值 0xA(=10) 和 0xB(=11)
```

```
解包只需两条基础位运算指令：
  1. 右移 4 位（shift right） → 取出高 nibble
  2. AND 掩码 0x0F           → 取出低 nibble
```

操作	**x86**	**ARM NEON**	**RISC-V RVV**	指令类型
右移	`_mm_srli_epi16`	`vshr_n_u8`	`vsrl.vx`	通用位移
AND 掩码	`_mm256_and_si256`	`vand_u8`	`vand.vx`	通用逻辑

这些都是 SSE2/NEON 时代就有的基础指令，RVV 同样原生支持。INT4 的全部"特殊之处"只在于存储时两个值塞一个 byte，解包时用现有位运算拆开即可。

MLAS 中 INT4 的实际解包代码（`q4gemm_avx512.cpp:124-132`）：

```
// 加载 packed INT4
const __m128i bvi4_0 = _mm_loadu_si128(b0ptr++);       // 16 bytes = 32 个 INT4

// 解包：shift + and
__m256i bytes0 = _mm256_set_m128i(_mm_srli_epi16(bvi4_0, 4), bvi4_0);  // 高低 nibble 拆
bytes0 = _mm256_and_si256(lowMask, bytes0);                              // mask 0x0F
// 现在 bytes0 有 32 个独立的 int8 值
```

5.2 INT4 MatMul 的实际计算路径：不是 INT8 指令，而是 FP32

MLAS 中 INT4 MatMuD 并没有走 **INT8 VNNI/SDOT** 指令路径。实际计算路径是解包后转成 FP32 做 `fmadd_ps`：

```
INT8 MatMul 路径：
  INT8 内存 ──→ VPDPBUSD (VNNI) ──→ int32 结果
  1 条专用指令完成

INT4 MatMul 实际路径（q4gemm_avx512.cpp）：
  INT4 packed ──→ shift+and 解包成 int8
                ──→ cvtepi8_epi16（int8 → int16）
                ──→ cvtepi16_epi32（int16 → int32）
                ──→ cvtepi32_ps（int32 → float32）
                ──→ mul_ps × scale（反量化）
                ──→ fmadd_ps（FP32 乘累加）
  约 10-12 条指令/每值
```

实际代码（`q4gemm_avx512.cpp:157-205`）：

```
// 类型转换链：int8 → int16 → int32 → float32
const __m256i vx16_lo0 = _mm256_cvtepi8_epi16(...);      // int8 → int16
__m512 bvf_lo0 = _mm512_cvtepi32_ps(                       // int32 → float
    _mm512_cvtepi16_epi32(vx16_lo0));                       // int16 → int32

// 反量化：乘以 block scale
bvf_lo0 = _mm512_mul_ps(bvf_lo0, _mm512_set1_ps(scale));

// FP32 乘累加
acc_lo0 = _mm512_fmadd_ps(bvf_lo0, av_lo, acc_lo0);       // FP32 FMADD
```

5.3 指令开销量化

指标	**INT8 MatMul**	**INT4 MatMul**	**INT4 / INT8**
加载字节数（16 个权重值）	16 bytes	8 bytes	**0.5x**（省一半）
LOAD 指令	1	1	1x
解包（shift + mask）	0	2-3	多 2-3 条
类型转换（int8→fp32）	0	4（int8→int16→int32→float→scale）	多 4 条
乘累加	**1 条 VNNI**	**1 条 FMADD (FP32)**	FP32 更慢
每值总指令数	**~2-3** 条	**~10-12** 条	**~4x**

INT4 MatMuD 每个权重值需要的指令数约为 INT8 的 **4** 倍，主要开销来自 int8→int16→int32→float 的类型转换链和 block scale 反量化。

5.4 内存带宽的含义

本文分析的"内存带宽"特指 **CPU** 后端的 **DRAM → L1/L2/L3 Cache** 搬运，不是 GPU 的显存搬运。

```
CPU 推理的存储层次：

  DRAM（主内存）            ← 权重存储在这里
    │  带宽 ~50 GB/s（典型 DDR4/DDR5）
    │  延迟 ~100ns
    ↓
```

```
  L3 Cache（共享）          ← 权重搬运到这里
    │  带宽 ~500 GB/s
    │  延迟 ~10ns
    ↓
  L2 / L1 Cache             ← 计算单元从这里取操作数
    ↓
  ALU（计算单元）           ← 等数据的时间 >> 计算时间（memory-bound 时）
```

Memory-bound 的本质：ALU 算得很快（GOPS 级别），但等数据从 DRAM 搬到 Cache 很慢。权重从 DRAM → Cache 的 DDR 带宽是瓶颈。

- INT8 权重 100MB → 需搬运 100MB 经过 DDR 总线
- INT4 权重 50MB → 只需搬运 50MB，省一半 **DDR** 带宽

5.5 INT4 MatMul 比 INT8 MatMuD 快还是慢？

取决于场景——核心是算术强度（**Arithmetic Intensity**）决定 **memory-bound vs compute-bound**：

```
算术强度 = FLOPs / Bytes_accessed

  算术强度低 → Memory-bound → 瓶颈在等数据 → INT4 省带宽有优势
  算术强度高 → Compute-bound → 瓶颈在 ALU → INT4 多 4x 指令反而更慢
```

场景 **1**：**LLM** 推理（**Memory-Bound**）—— **INT4** 赢

```
典型参数：权重 4096×4096, batch=1, M=1
  FLOPs = 2 × 1 × 4096 × 4096 = 33M
  INT8 权重体积 = 16.8MB
  算术强度 ≈ 2.0 → Memory-bound

INT8: 等待 DRAM→Cache 100ms + 计算 10ms = 110ms
INT4: 等待 DRAM→Cache  50ms + 计算 40ms =  90ms  ← 快！
       (省了一半带宽)        (指令多了4倍)
```

场景 **2**：**CNN FC** 层 **/** 大 **batch MatMul**（**Compute-Bound**）—— **INT8** 赢

```
典型参数：batch=32, M=32, K=4096, N=4096
  FLOPs = 2 × 32 × 4096 × 4096 = 1.07G
  INT8 权重体积 = 16.8MB（B 矩阵可被 32 行复用）
  算术强度 ≈ 60 → Compute-bound

INT8: 等待内存 5ms + 计算 50ms = 55ms
INT4: 等待内存 3ms + 计算 200ms = 203ms  ← 慢 3.7x！
       (省了点带宽)   (指令多了4倍)
```

MatMul 场景	算术强度	瓶颈	谁快
LLM linear (batch=1, 权重大)	~2	DRAM→Cache 带宽	**INT4 快 1.2-1.5x**
LLM linear (batch≥32)	~60+	ALU 计算	**INT8 快 2-4x**
CNN FC 层（权重小）	~50+	ALU 计算	**INT8 快 3-4x**
YOLOv11 检测头 MatMul	~100+	ALU 计算	**INT8 快 3-4x**

结论：**INT4 MatMuD** 在 **CPU** 后端仅在 **LLM** 单 **batch** 推理（**memory-bound**）时有优势。对 **YOLOv11** 等计算密集场景，**INT8 MatMul** 明显更快（**3-4x**），因为 **INT4** 多出的 **4** 倍指令开销无法被带宽节省抵消。

六、CPU 后端 INT8 Kernel 详解

6.1 x86 INT8 Kernel 分发链

```
优先级从高到低：
AMX-INT8          → MlasGemmU8S8DispatchAmx          (矩阵引擎, 16×16 tile)
  ↓
AVX512-VNNI       → MlasGemmU8S8KernelAvx512Vnni     (512-bit, _mm512_dpbusd_epi32)
  ↓
AVX512-Core       → MlasGemmU8S8KernelAvx512Core      (512-bit, 通用 SIMD 组合)
  ↓
AVX2-VNNI         → MlasGemmU8S8KernelAvxVnni         (256-bit, AVX-VNNI 指令)
  ↓
AVX2+FMA          → MlasGemmU8S8DispatchAvx2           (256-bit, _mm256_maddubs_epi16 组合
  ↓
SSE4.1            → MlasGemmU8S8DispatchSse41           (128-bit, _mm_maddubs_epi16 + _mm_
  ↓
SSE2              → MlasGemmU8X8DispatchSse             (128-bit, 纯手工拆分)
```

关键文件：- `onnxruntime/core/mlas/lib/qgemm_kernel_sse.cpp` - `onnxruntime/core/mlas/lib/qgemm_kernel_sse41.cpp` - `onnxruntime/core/mlas/lib/qgemm_kernel_avx2.cpp` - `onnxruntime/core/mlas/lib/amd64/QgemmU8X8KernelAvx2.asm` - `onnxruntime/core/mlas/lib/qdwconv.cpp`（量化 DWConv） - `onnxruntime/core/mlas/lib/convsym.cpp`（对称量化 Conv）

支持的量化格式：U8S8, U8U8, S8S8, S8U8

6.2 ARM INT8 Kernel 分发链

```
优先级从高到低：
SMMLA/UMMLA (ARMv8.6 I8MM) → MlasGemmS8S8DispatchSmmla    (矩阵级 8×8 乘累加)
  ↓
SDOT/UDOT   (ARMv8.4 Dot)  → MlasGemmS8S8DispatchSdot      (4 元素点积)
  ↓
NEON 基线   (ARMv8.0)      → 手工模拟 dot product           (逐元素乘加)
```

关键文件：- `onnxruntime/core/mlas/lib/qgemm_kernel_neon.cpp` - `onnxruntime/core/mlas/lib/qgemm_kernel_sdot.cpp` - `onnxruntime/core/mlas/lib/qgemm_kernel_smmla.cpp` - `onnxruntime/core/mlas/lib/qgemm_kernel_ummla.cpp` - `onnxruntime/core/mlas/lib/aarch64/QgemmS8S8KernelSdot.S` - `onnxruntime/core/mlas/lib/aarch64/QgemmS8S8KernelSmmla.S`

平台检测：

```
HasArmNeonDot()     → 检测 ARMv8.2+ Dot Product
HasArmNeon_I8MM()   → 检测 ARMv8.4+ I8MM (SMMLA/UMMLA)
HasArmSVE()         → 检测 SVE 支持
```

6.3 RISC-V INT8 Kernel 状态

完全缺失。MLAS 中没有以下任何内容：- 无 `MLAS_TARGET_RISCV` 宏定义 - 无 `qgemm_kernel_rvv.cpp` 或类似文件 - 无 RVV intrinsic（`__riscv_v*`）调用 - 无 RISC-V 平台特征检测 - 无汇编 kernel（`.S` 文件）

编译后 所有 **INT8** 算子走 **scalar C fallback**，无任何向量加速。

现有 **RISC-V** 基础设施：- `cmake/riscv64.toolchain.cmake`：仅配置交叉编译器路径 - `tools/scripts/build_riscv64.sh`：下载 RISC-V GNU Toolchain + QEMU 模拟 - 没有启用任何 RVV 编译选项

七、CPU 后端 INT4 Kernel 详解

7.1 x86 INT4 Kernel

Kernel	架构要求	**4-bit** 格式	关键文件
SQNBitGemm	AVX512	BlkQ4Sym, BlkQ4Zp8 (block=16/32/64/128)	`sqnbitgemm_kernel_avx512.cpp` + `*_blklen*.h`

Kernel	架构要求	4-bit 格式	关键文件
Q4Gemm	AVX512	BlkQ4Sym64, BlkQ4Sym128	`q4gemm_avx512.cpp`
MatMulBnb4	通用 C	NF4, FP4	`contrib_ops/cpu/quantization/matmul_bnb4.cc`
MatMulNbits	通用 C	N-bit 通用	`contrib_ops/cpu/quantization/matmul_nbits.cc`

7.2 ARM INT4 Kernel

Kernel	架构要求	关键文件
SQNBitGemm NEON	ARMv8.2+	`sqnbitgemm_kernel_neon_int8.cpp`
SQNBitGemm NEON+I8MM	ARMv8.4+ (I8MM)	`sqnbitgemm_kernel_neon_int8_i8mm.cpp`
SQNBitGemm NEON FP32	ARMv8.0+	`sqnbitgemm_kernel_neon_fp32.cpp`
QNBitGemm	NEON 基线	`qnbitgemm_kernel_neon.cpp`

7.3 RISC-V INT4 Kernel

完全缺失，同 INT8。

八、GPU 后端量化支持

8.1 CUDA INT8

算子	实现	依赖
MatMulInteger	`cuda/math/matmul_integer.cu`	cuBLAS `cublasGemmEx` (CUDA_R_8I → CUDA_R_32I)
QuantizeLinear		自定义 CUDA kernel

算子	实现	依赖
	`cuda/tensor/quantize_linear.cu`	
DequantizeLinear	`cuda/tensor/`	自定义 CUDA kernel

架构要求：通过 cuBLAS 自动适配，无需手动指定 compute capability。

8.2 CUDA INT4

Kernel	架构要求	**4-bit** 格式	关键文件
MatMul4Bits	sm_53+	标准 INT4 (packed uint32)	`contrib_ops/cuda/quantization/matmul_4bits.cu`
MatMulBnb4	sm_80+	NF4, FP4	`contrib_ops/cuda/quantization/matmul_bnb4.cu`
DequantizeBlockwise4Bits	sm_53+	Block-wise 4-bit	`contrib_ops/cuda/quantization/dequantize_blockwise_4bits.cu`
FP16+INT4 混合精度	sm_80+	FP16+INT4 scale/zeros	`contrib_ops/cuda/llm/fpA_intB_gemm/`
BF16+INT4 混合精度	sm_80+	BF16+INT4 scale/zeros	同上

8.3 TensorRT INT8

- 通过 `trt_int8_enable` 开关启用
- 支持校准表（`int8_calibration_cache_name`）
- 需硬件 `platformHasFastInt8()` 通过
- DLA 支持 INT8 模式

九、YOLOv11 量化推理分析

9.1 YOLOv11 关键算子与 INT8 支持状态

YOLOv11 算子	CPU INT8	CUDA INT8	备注
Conv (标准)	✅ QLinearConv	✅ cuBLAS INT8	完整支持
DWConv (group=cin)	✅ QLinearConv (优化路径)	✅ cuBLAS	DW 有专用 kernel `qdwconv.cpp`
BatchNorm	✅ int8_t 模板实例	✅	融合到 Conv 后通常无开销
SiLU	❌ 无量化版本	❌	必须 **dequant→FP32→quant**
Sigmoid	✅ QLinearSigmoid (LUT)	❌	contrib_ops 中有实现，用查找表
Concat	⚠️ 无专用 INT8 kernel	⚠️	通用类型支持，可能 memcpy
Resize (upsample)	✅ 有 int8_t kernel	⚠️	CPU 有优化
Split	⚠️ 无专用 INT8 kernel	⚠️	通用支持
NMS / EfficientNMS	❌ 仅 FP32	❌	检测头必须 **FP32**
Transpose/ Reshape/ Squeeze	⚠️ 无专用 INT8 kernel	⚠️	纯 shape 操作，无计算

9.2 YOLOv11 INT8 推理流水线

```
[Conv Block]                    [Conv Block]                    [Detect Head]
Conv(DW+PW) ──✅──→             Conv ──✅──→                    Conv ──✅──→
BN(fused)   ──✅──→             BN(fused) ──✅──→               BN ──✅──→
```

```
SiLU        —❌—→ DQ→FP32→Q     SiLU —❌—→ DQ→FP32→Q          SiLU —❌—→
                                  ↓                              ↓
                               Concat —⚠️—→                   NMS —❌—→ FP32
                               Upsample —✅—→                    ↓
                                                            输出（FP32）
```

9.3 各架构跑 YOLOv11 INT8 的预期

架构	**Conv INT8**	**DWConv INT8**	**SiLU 开销**	整体加速比
x86 (VNNI)	✅ 高效	✅ 有专用 kernel	DQ/Quant	**2-4x**
x86 (AMX)	✅ 矩阵引擎	✅ 有专用 kernel	DQ/Quant	**4-8x**
ARM (SDOT)	✅ 高效	✅ 有专用 kernel	DQ/Quant	**2-3x**
ARM (SMMLA)	✅ 矩阵级	✅ 有专用 kernel	DQ/Quant	**3-5x**
RISC-V	❌ scalar	❌ scalar	scalar	**~1x (无加速)**

十-A、为什么只有部分算子做了 INT8 量化

10A.1 INT8 加速的前提条件：MAC 密集型算子

INT8 之所以能比 FP32 快 2-4×，是因为硬件提供了 **INT8** 乘累加（**MAC**）的专用高吞吐路径：

架构	**FP32 MAC** 吞吐	**INT8 MAC** 吞吐	倍率
x86 VNNI	32 FLOP/cycle	128 INT8-OP/cycle	**4×**
ARM SDOT	8 FLOP/cycle	32 INT8-OP/cycle	**4×**

只有 **MAC** 密集型算子才能吃到这个 **4×** 红利。 对于 memory-bound 或非 MAC 型算子，INT8 几乎没有收益。

10A.2 YOLOv11 各算子的 FLOPs 占比与 INT8 收益分析

```
YOLOv11n 推理一次的总计算量分布：

Conv/DWConv:      ~2.8 GFLOPs   ← 95%+ 的计算量，compute-bound    INT8 收益巨大（3-4×）
MatMul (detect):  ~0.1 GFLOPs   compute-bound                     INT8 收益大
BN:               融合进 Conv，零开销                             不需要独立 kernel
SiLU:             ~0.05 GFLOPs  ← < 2% 的计算量，memory-bound     INT8 几乎没收益（< 10%）
Sigmoid:          ~0.001 GFLOPs ← < 0.1%，memory-bound            无所谓
Concat:           0 GFLOPs      纯内存搬运                        INT8 等于 memcpy
NMS:              ~0.0001 GFLOPs ← 排序+比较，非 MAC 型           INT8 完全不适用
Upsample:         ~0.01 GFLOPs  ← 微小，memory-bound              INT8 收益 < 5%
```

10A.3 各类算子的详细分析

1. Conv / MatMul —— 做了有大收益

```
Conv 3×3, 640×640, 64→128 channels:
  MAC 次数 = 640 × 640 × 3 × 3 × 64 × 128 = 28.7 亿次
  算术强度 ≈ 1100 → Compute-bound
  INT8 收益：纯计算 4× 加速 → 整体 3-4× 提升 ✅
```

2. SiLU / HardSwish —— 做了几乎没收益，且精度有损

```
SiLU on 640×640×128 feature map:
  计算量 = 52.4M 次逐元素操作
  内存访问 = ~100MB
  算术强度 ≈ 1 → Memory-bound

  瓶颈在搬运数据，不在计算。INT8 的 MAC 吞吐优势完全用不上。
  FP32 SiLU: ~0.2ms（等内存）
  INT8 SiLU: ~0.18ms（还是等内存）
  收益: < 10%，几乎可忽略
```

精度问题：SiLU = `x × sigmoid(x)` - 无界输入 `x` × 有界 sigmoid 输出 [0,1]，动态范围极不均匀 - 量化误差在乘法中被放大 - 不同通道的 SiLU 激活分布差异大 → per-tensor scale 精度不够

结论：收益 **≈ 0**，代价 **=** 精度下降 **+** 实现复杂度。

3. NMS —— 做了没意义

```
NMS 操作：排序 + IoU 计算 + 阈值比较
  MAC 密度：几乎为零
  INT8 MAC 优势完全不适用
```

```
  BBox 坐标差 1 pixel 可能导致检测结果变化
  → 所有框架均保持 FP32
```

4. BatchNorm —— 不需要做

```
BN 参数融合进 Conv：W' = γ/σ × W, b' = γ/σ × (b - μ) + β
融合后 BN 消失，不需要独立的 INT8 kernel
```

5. Concat / Split / Transpose —— 无计算收益

```
纯内存操作（memcpy / shape 变换），没有任何 MAC 计算
INT8 vs FP32 的区别仅是搬运数据量减半（4B → 1B）
但瓶颈在内存带宽，不在数据类型
```

10A.4 跨框架对比：其他推理框架怎么处理？

框架	**SiLU INT8**	**Sigmoid INT8**	**HardSwish INT8**	**NMS INT8**	策略
TensorRT	隐式量化融合 PWN；显式 QDQ 精度下降	升 FP16 计算	声称支持，实际走 FP	FP32 only	混合精度
TFLite	无原生 op；拆为 LOGISTIC+MUL	有（LOGISTIC op）	不在 INT8 列表	FP32 only	拆分+混合精度
OpenVINO	通过 NNCF 排除，保持 FP32	可排除	可排除	FP32 only	选择性排除
NCNN	**LUT** 查表实现	LUT	LUT	FP32 only	LUT 逼近
TVM	保持高精度	保持高精度	保持高精度	FP32 only	QDQ 混合精度
SNPE/QNN	需自定义 UDO	支持（HTP）	未文档化	FP32 only	混合精度

NCNN 是唯一直接提供 **INT8 SiLU** 的框架，采用 LUT 方式。但这是"够用"精度，不保证所有场景。学术界已有 I-BERT（arXiv:2101.01321）验证 LUT 方案可行，精度损失 0.5-1% mAP。

10A.5 标准混合精度管线与开销

```
YOLOv11 INT8 推理的实际管线（所有框架通用）：

Conv/DWConv  ──✅ INT8──→  (占 95%+ FLOPs, 3-4× 加速)
BN           ──✅ 融合──→  (参数融入 Conv, 零开销)
SiLU         ──❌ DQ→FP32→SiLU→Q──→  (memory-bound, INT8 无收益)
Concat       ──✅ INT8──→  (带 scale 对齐的 memcpy)
NMS          ──❌ FP32──→  (非 MAC 型, 精度敏感)
```

DQ/Q 转换开销： - 单次 DQ/Q：约 2-4 条 SIMD 指令（scale 乘法 + zero-point 加法 + clip） - YOLOv11n 约 40-50 个 Conv 块 → 40-50 次 DQ+Q 转换 - 总推理 ~5ms（VNNI）→ DQ/Q 总开销 ~1-2ms → 约 **20-30%** 的额外开销

10A.6 行业趋势

方向	进展
NMS-free 架构	YOLOv10 已推出 NMS-free 变体，消除 NMS 的 FP32 瓶颈
LUT 逼近	NCNN/I-BERT 已验证 LUT 方案，精度损失 0.5-1% mAP
混合精度自动搜索	OpenVINO NNCF / TensorRT 自动选择每层最优精度
QAT（量化感知训练）	训练时模拟量化误差，让模型适应 → SiLU 附近权重分布更量化友好

十、RISC-V RVV 补全方案

10.1 需要实现的 MLAS Kernel

Kernel	功能	**RVV** 指令组合	注意事项
QGemm S8S8	INT8 矩阵乘	外积法：`vle8.v` + `vwmacc.vv`	优先用外积避免 vredsum 瓶颈

Kernel	功能	RVV 指令组合	注意事项
QGemm U8S8	混合符号 INT8 GEMM	同上 + 符号翻转预处理	—
QDWConv S8S8	INT8 深度可分离卷积	`vle8.v` + `vslide` + `vwmacc`	逐 channel 独立，天然适合 RVV
QConv Sym	对称量化卷积	`vwmacc.vv` (widening)	—
Dequantize	INT8→FP32 反量化	`vfwcvt.f.x.v` + `vfmacc.vf`	—
Quantize	FP32→INT8 量化	`vfncvt.x.f.v` + clip	—

10.2 MLAS 集成步骤

```
1. mlas.h              → 添加 MLAS_TARGET_RISCV64 宏
2. platform.cpp        → 添加 RISC-V 平台检测（__riscv_v 特征检测）
3. 新增文件：
   qgemm_kernel_rvv.cpp        → INT8 GEMM (RVV intrinsic, 外积法)
   qdwconv_kernel_rvv.cpp      → INT8 DWConv
   qconv_sym_kernel_rvv.cpp    → 对称量化卷积
4. qgemm.h             → 分发逻辑添加 #elif defined(MLAS_TARGET_RISCV64)
5. CMakeLists.txt      → 编译条件添加 RISC-V 源文件
```

10.3 RVV GEMM Kernel 设计要点

方案 **A**：外积法（**Outer Product**）—— **RVV** 现有指令可用

```
// 外积法 C[M][N] += A[M][K] × B[K][N]
// 取 A 的一个元素 A[i][k] 和 B 的一行 B[k][0..N-1]
// 外积：A[i][k] * B[k][j] → 累加到 C[i][j]
// 这样 C 的每个元素独立更新，不需要横向规约

vint8m1_t b_row = vle8_v_i8m1(B_row_ptr, vl);      // 加载 B 的一行（N 个 int8）
int8_t a_val = A[i * K + k];                          // 取 A 的一个元素
vint16m2_t ext_b = vwadd_vx_i16m2(b_row, 0, vl);    // int8 → int16
vint32m4_t prod = vwmul_vx_i32m4(ext_b, a_val, vl);  // int16 × int8 → int32
vadd_vv_i32m4(C_row_acc, C_row_acc, prod, vl);        // 累加到 C 行
// C 行的 N 个元素全部并行更新，一次 vle8 + vwmul + vadd
// 最终 vse32 一次写回整个 C 行 ✅ 全 SIMD
```

外积法回避了 **vredsum** 瓶颈：C 的每个元素由多次外积累加得到，不需要横向规约。代价是： - 每个 A 元素需要单独 broadcast，计算密度低于融合 dot 指令 - 需要 LMUL>1 才能同时展开多行 - 预估吞吐为 ARM SDOT 的 **30-50%**

方案 **B**：内积法 **+** 自定义 **`vsegdot`** 指令 **——** 推荐方案

若有自定义 `vsegdot` 指令，可采用与 x86 VNNI / ARM SDOT 完全相同的内积 GEMM 微内核架构：

```
// 内积法：与 ARM SDOT kernel 同构
// A 已按 CopyPackA 交错打包，B 已按 CopyPackB 交错打包
// packed A: [A0 A1 A2 A3 B0 B1 B2 B3 C0 C1 C2 C3 D0 D1 D2 D3] (16B)
// packed B: [b0 b1 b2 b3 | b4 b5 b6 b7 | ... | b12 b13 b14 b15] (16B, 4行交错)

vint8m1_t a_vec = vle8_v_i8m1(A_ptr, 16);           // 加载 packed A（4行×4K）
vint8m1_t b_vec = vle8_v_i8m1(B_ptr, 16);           // 加载 packed B（4列×4K行）
vint32m1_t c_acc = vsegdot_vv_i32m1(c_acc, a_vec, b_vec, 4);  // 分段点积！
// c_acc[0] += a[0]*b[0] + a[1]*b[1] + a[2]*b[2] + a[3]*b[3]  → C[0]
// c_acc[1] += a[4]*b[4] + a[5]*b[5] + a[6]*b[6] + a[7]*b[7]  → C[1]
// c_acc[2] += a[8]*b[8] + a[9]*b[9] + a[10]*b[10] + a[11]*b[11] → C[2]
// c_acc[3] += a[12]*b[12] + a[13]*b[13] + a[14]*b[14] + a[15]*b[15] → C[3]
vse32_v_i32m1(C_ptr, c_acc, 4);                     // 4 个 i32 一次写回 ✅
```

内积法 **+** **`vsegdot`** 的优势： - 与 x86/ARM kernel 同构，可直接复用 MLAS 的 CopyPackA/CopyPackB 逻辑 - 一条指令同时推进 4 个 C 元素的更新，计算密度与 ARM SDOT 相当 - 无需 `vredsum`，无串行规约瓶颈 - 预估吞吐为 ARM SDOT 的 **85-100%**

两种方案的总结对比

	方案 **A**：外积（现有 **RVV**）	方案 **B**：内积 **+** **`vsegdot`**（自定义）
需要的指令	`vle8`，`vwmacc`，`vadd` (已有)	`vsegdot` (需自定义)
PackB 交错打包	不需要（直接用行主序）	需要（同 x86/ARM 交错格式）
C 向量化输出率	12.5%（逐行产出）	100%（8×8 块全在寄存器）
vs ARM SDOT 性能	30-50%	85-100%
实现复杂度	低（从头设计）	

	方案 **A**：外积（现有 **RVV**）	方案 **B**：内积 **+** `vsegdot` （自定义）
		中（移植 MLAS kernel 模式）

十一、结论

1. **x86** 和 **ARM** 的 **INT8** 推理支持已经非常成熟，从标准 SIMD 到专用指令（VNNI/AMX/SDOT/SMMLA）形成完整梯度，且自动运行时分发到最优 kernel。

2. **INT4** 在 **CPU** 和 **GPU** 后端均无卷积算子，只有 MatMul 路径，专门服务 LLM 场景。YOLOv11 无法走 INT4 推理（没有 QLinearConv 的 INT4 版本）。

3. **INT4** 不是**"**更快的 **INT8"**，而是"用更多计算换更少内存"的存储优化：

4. 解包不需要新指令，仅需通用的 `shift + and` 位运算
5. MLAS 中 INT4 MatMul 实际走 FP32 计算路径（int8→int16→int32→float→fmadd），每值约 10-12 条指令，是 INT8（~2-3 条 VNNI）的 **4** 倍

6. 仅在 memory-bound 场景（LLM 单 batch）有优势；对 CNN/YOLOv11 等计算密集场景，INT8 快 3-4 倍

7. 只有 **MAC** 密集型算子值得做 **INT8**：Conv/DWConv/MatMul 占 YOLOv11 推理 95%+ 的 FLOPs，是 compute-bound 的，INT8 的 MAC 吞吐 4× 优势能充分发挥。其余算子不做 INT8 不是因为"做不了"，而是做了没收益：

8. **SiLU / HardSwish**：逐元素操作，memory-bound，算术强度≈1，INT8 vs FP32 差异 < 10%。精度还有额外损失。
9. **NMS**：排序+比较操作，非 MAC 型，INT8 加速机制完全不适用，且 BBox 精度敏感。
10. **BN**：参数直接融合进 Conv 权重，不需要独立 kernel。
11. **Concat / Split / Transpose**：纯内存搬运，零 MAC 计算。

12. 这是所有主流框架（TensorRT / TFLite / OpenVINO / NCNN / TVM / SNPE）的共同选择。

13. **RVV** 最大的架构劣势是缺少分段规约：x86 VNNI 一条指令可从一对向量寄存器产出 8-16 个独立 int32 结果，ARM SDOT 产出 4 个；RVV 的 vredsum 只能产出 1 个标量，导致 C 的写回变成串行。

14. **INT8 GEMM** 的效率差异源于三层耦合操作：① PackB 交错打包（`vpunpck*` / `vzipq`，RVV 无高效等价）；② 融合分段点积（`vpdpbusd` / `sdot`，RVV 无等价）；③ 4×4/4×8 矩阵微内核展开。三层都依赖分段规约能力。

15. 外积法是 **RVV** 的可行 **fallback**，但预估性能仅为 ARM SDOT 的 30-50%。核心瓶颈是每个 A 元素需单独 broadcast，C 输出逐行产出，计算密度低于内积路径。

16. 自定义 **`vsegdot`** 指令是关键突破口：增加分段点积指令后，可采用与 x86/ARM 同构的内积 GEMM 微内核，预估性能提升 2-3×（从 ARM SDOT 的 30-50% 提升到 85-100%）。该指令的语义为：4×i8 乘加→1×i32，同时保留多个独立 lane 并行输出。

17. **YOLOv11** 的 **INT8** 推理在 **x86/ARM** 上可行，Conv/DWConv 有完整优化 kernel，SiLU 处走 DQ→FP32→Q 混合精度路径，额外开销约 20-30%。

18. **RISC-V** 是 **ONNX Runtime** 量化生态的空白区域：零个优化 kernel、无平台检测、仅有交叉编译 toolchain。FP32 也同样没有 RVV 实现。这是 RVV 1.0 填补空白的巨大机会，且 **INT8 kernel** 是唯一需要优先做好的目标，而自定义 `vsegdot` 指令是解锁高性能 INT8 推理的核心前提。